

# Margin Ledger: Predicting Corporate Profit Margin from ESG and Financial Data

---

Student Name — Class/Section — August 2026

- [1. Abstract / Executive Summary](#)
- [2. Problem Statement](#)
- [3. Dataset Description](#)
- [4. Exploratory Data Analysis](#)
- [5. Data Cleaning](#)
- [6. Categorical → Numerical Encoding](#)
- [7. Preprocessing](#)
- [8. Train/Test Split](#)
- [9. Model Development](#)
- [10. Evaluation](#)
- [11. Final Model & Claude Artifact / Connector / skills.md](#)
- [12. Deployment](#)
- [13. Conclusion & Future Work](#)
- [14. References](#)

## 1. Abstract / Executive Summary

---

This project investigates whether a company’s sustainability profile (Environmental, Social, and Governance scores) helps predict its financial performance. Using a panel dataset of 10,000 company-years covering 1,000 companies across 9 industries and 7 regions (2015–2025), a regression pipeline was built to predict **profit margin** from financial scale (revenue, market cap, growth), sector/region, and ESG sub-scores. Four models were trained and compared; a **Gradient Boosting Regressor** was selected as the best performer (test RMSE 6.62 percentage points,  $R^2 = 0.44$ ). The dominant predictors are financial scale (market capitalization and revenue), with ESG scores contributing a smaller but non-trivial share of predictive power — notably, the **Environmental** sub-score correlates with profit margin more than the Social or Governance sub-scores do. The final pipeline was deployed as a static, client-side web application (“Margin Ledger”) suitable for Netlify hosting, with an interpretable Ridge Regression model ported to JavaScript and numerically verified against the Python pipeline.

## 2. Problem Statement

---

**Business question:** does sustainability performance (“ESG”) help explain or predict a company’s profitability, alongside conventional financial and categorical factors?

**Task type:** supervised regression.

**Target variable:** ProfitMargin (%), continuous.

**Input features:** Industry (categorical, 9 levels), Region (categorical, 7 levels), Year, Revenue, MarketCap, GrowthRate, ESG\_Overall, ESG\_Environmental, ESG\_Social, ESG\_Governance,

`CarbonEmissions`, `WaterUsage`, `EnergyConsumption`.

**Why it matters:** investors, regulators, and corporate strategists increasingly weigh ESG performance in capital allocation decisions. A model that quantifies the relationship between ESG scores and profitability (even an imperfect one) is a useful starting point for that conversation, and a realistic exercise in the kind of “does X predict Y, controlling for other factors” question common in applied business analytics.

### 3. Dataset Description

---

- **Source:** ESG & Financial Performance Dataset, published on Kaggle by Shriyash Jagtap.
- **URL:** <https://www.kaggle.com/datasets/shriyashjagtap/esg-and-financial-performance-dataset>
- **Access date:** August 2026.
- **Size:** 11,000 rows × 16 columns as published (1,000 companies × 11 years, 2015–2025); 10,000 rows after cleaning (see §5).
- **License:** per the Kaggle dataset page (open for research/educational use — see page for exact terms).
- **Nature of the data:** this dataset is explicitly **synthetic but realistic** — it simulates ESG and financial metrics for 1,000 companies rather than scraping verified filings from real public companies. This is disclosed transparently rather than presented as real-company data. The choice is defensible for a course project: the dataset is recent (updated 2025), well-structured, free of privacy/licensing complications, and large enough (10,000 usable rows) to support a real train/test regression workflow with meaningful signal-to-noise characteristics, as the results in §10 demonstrate.
- **Data dictionary:** see README.md §2 for the full feature dictionary (column name, type, description).

### 4. Exploratory Data Analysis

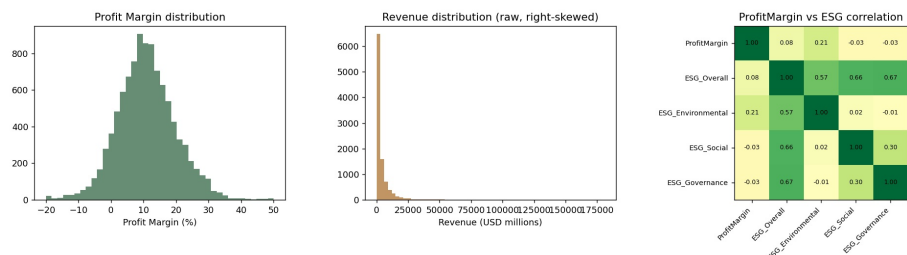
---

**Shape:** 11,000 rows × 16 columns, confirmed as a balanced panel: 1,000 unique `CompanyID` values, each with exactly 11 rows (one per year, 2015–2025), no duplicate (`CompanyID`, `Year`) pairs.

**Distributions:** `ProfitMargin` and `GrowthRate` are roughly symmetric (skewness 0.15 and -0.08 respectively), ranging from -20% to 50% and -36% to 38% respectively — realistic given that ~10% of company-years show negative margins (loss-making periods). By contrast, `Revenue`, `MarketCap`, `CarbonEmissions`, `WaterUsage`, and `EnergyConsumption` are **heavily right-skewed** (skewness 7–16), consistent with the well-known fact that company size distributions have a long tail of very large firms.

**Category balance:** both categorical features are reasonably balanced — `Industry` ranges from 1,045 rows (Technology) to 1,331 rows (Healthcare) across 9 categories; `Region` ranges from 1,463 (Africa) to 1,672 (Asia) across 7 categories. No category is so rare that one-hot encoding would create a near-empty column.

**ESG vs profit margin correlation** (Figure 1, right panel): `ProfitMargin` correlates weakly with `ESG_Overall` ( $r = 0.08$ ), more meaningfully with `ESG_Environmental` ( $r = 0.21$ ), and is essentially uncorrelated with `ESG_Social` ( $r = -0.03$ ) and `ESG_Governance` ( $r = -0.03$ ). This is an important early finding: if ESG “pays off” in this dataset, it appears to do so mainly through the Environmental dimension, not Social or Governance — a nuance that a single composite `ESG_Overall` score would have hidden.



EDA overview: profit margin distribution, revenue skew, and ESG correlation heatmap

## 5. Data Cleaning

| Issue                                              | Finding                                                               | Action                   | Rationale                                                                                                                                                      |
|----------------------------------------------------|-----------------------------------------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Missing values                                     | <code>GrowthRate</code> null for exactly 1,000 rows, all in Year=2015 | Dropped these 1,000 rows | Structural missingness (no prior year exists for a first-year growth calculation), not random — imputing would fabricate a value with no true prior-year basis |
| Duplicate rows                                     | 0 found                                                               | None needed              | —                                                                                                                                                              |
| Duplicate (CompanyID, Year)                        | 0 found                                                               | None needed              | —                                                                                                                                                              |
| Out-of-range ESG scores (outside 0–100)            | 0 found                                                               | None needed              | —                                                                                                                                                              |
| Non-positive Revenue/MarketCap/Carbon/Water/Energy | 0 found                                                               | None needed              | These should always be positive; confirmed clean                                                                                                               |
| CompanyID ↔ CompanyName consistency                | Max 1 name per ID                                                     | None needed              | Confirms no ID reuse/corruption                                                                                                                                |

**Before** → **after**: 11,000 rows × 16 columns → 10,000 rows × 16 columns. No columns were dropped at the cleaning stage (identifier columns are excluded later, at the feature-selection step, not during cleaning).

## 6. Categorical → Numerical Encoding

`Industry` (9 levels) and `Region` (7 levels) were converted using **one-hot encoding** (`sklearn.preprocessing.OneHotEncoder`, `handle_unknown="ignore"`), producing 16 additional binary columns. One-hot encoding was chosen over ordinal encoding because neither `Industry` nor `Region` has a natural order — assigning integers (e.g. Africa=0, Asia=1, ...) would falsely imply a ranking that a linear or distance-based model could exploit. With only 9 and 7 categories respectively, and no rare categories, the dimensionality cost of one-hot encoding is small (16 extra columns) and well justified.

## 7. Preprocessing

- **Skew correction:** `Revenue`, `MarketCap`, `CarbonEmissions`, `WaterUsage`, and `EnergyConsumption` were log1p-transformed (`log(1+x)`) before scaling, reducing their skewness from

7–16 down to near-symmetric. This matters for the linear/Ridge models, which are sensitive to feature scale and skew; tree-based models are invariant to monotonic transforms but were run through the same pipeline for consistency and fair comparison.

- **Scaling:** all numeric features (including the log-transformed ones) were standardized to zero mean / unit variance using `sklearn.preprocessing.StandardScaler`.
- **Imbalance:** not applicable — this is a regression task, not classification, so class-imbalance handling does not apply. The continuous target’s distribution was inspected instead (§4) and found reasonably well-behaved.
- **Reproducible pipeline:** encoding and scaling are wrapped in a single `sklearn.compose.ColumnTransformer` inside a `sklearn.pipeline.Pipeline` with the model as the final step (`src/preprocess.py`, `src/train.py`). This guarantees that calling `.fit()` on the pipeline with training data only fits the scaler/encoder statistics on training data — test data preprocessing statistics are never leaked into training.

## 8. Train/Test Split

**Critical design decision — grouped split.** This dataset is a panel: each of the 1,000 companies contributes 10 rows (one per year, 2016–2025, after the 2015 rows were dropped in cleaning). A naive **row-wise** random 80/20 split would place different years of the *same company* into both the training and test sets. Since a model can partially learn company-specific quirks from other years of that same company, this would leak information and inflate apparent test performance.

To prevent this, the split was performed with `sklearn.model_selection.GroupShuffleSplit`, grouping rows by `CompanyID`, so that **all 10 years of any given company fall entirely into either train or test, never both**. This was verified programmatically: the intersection of train-set and test-set `CompanyID` values is empty (0 companies in common).

- **Split proportions:** 80% train (800 companies, 8,000 rows) / 20% test (200 companies, 2,000 rows).
- **Cross-validation:** in addition to the held-out test set, 5-fold cross-validation (`KFold`, shuffled) was run on the training set only, to check that test-set performance wasn’t a fluke of one particular split.

## 9. Model Development

Four regression algorithms were trained through the identical preprocessing pipeline, so that the only thing varying between them is the model itself:

| Model             | Key hyperparameters                             | Why it was included                                                                                                                                   |
|-------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linear Regression | none (OLS)                                      | Simplest possible baseline — establishes a performance floor                                                                                          |
| Ridge Regression  | alpha = 1.0                                     | Regularized linear model; also the model <b>deployed</b> to the static web app, since its inference can be exactly reproduced in JavaScript (see §11) |
| Random Forest     | 300 trees, max_depth=10, min_samples_leaf=5     | Captures non-linear interactions (e.g. ESG’s effect on margin may differ by industry) without needing manual interaction terms                        |
| Gradient Boosting | 300 estimators, max_depth=3, learning_rate=0.05 | Typically the strongest tabular-data performer among tree ensembles; included as the primary comparison                                               |

|  |  |                         |
|--|--|-------------------------|
|  |  | to the linear baselines |
|--|--|-------------------------|

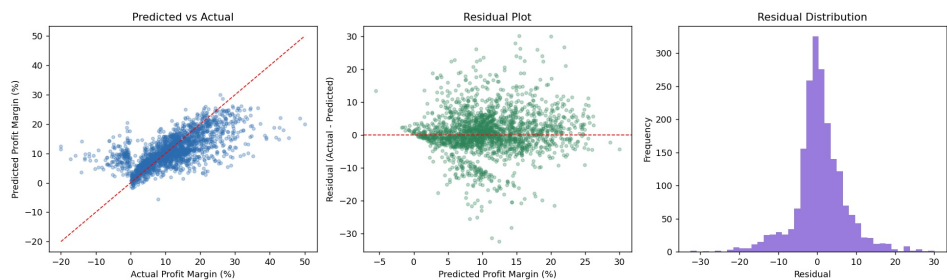
Random Forest and Gradient Boosting were chosen over, e.g., a plain Decision Tree or SVM because this is a **medium-sized, mixed categorical/numeric tabular problem** where ensembles of trees reliably outperform single trees and typically match or beat kernel methods (SVR) at a fraction of the tuning effort — a well-established pattern for this data regime, not a choice made because these algorithms are popular.

## 10. Evaluation

| Model                               | Test MAE | Test RMSE   | Test R <sup>2</sup> | 5-fold CV RMSE (train) |
|-------------------------------------|----------|-------------|---------------------|------------------------|
| <b>Gradient Boosting (selected)</b> | 4.55     | <b>6.62</b> | <b>0.441</b>        | 6.15 ± 0.24            |
| Random Forest                       | 4.42     | 6.89        | 0.395               | 5.57 ± 0.16            |
| Ridge (deployed)                    | 4.73     | 6.89        | 0.395               | 7.03 ± 0.21            |
| Linear Regression                   | 4.76     | 6.90        | 0.393               | 7.03 ± 0.20            |

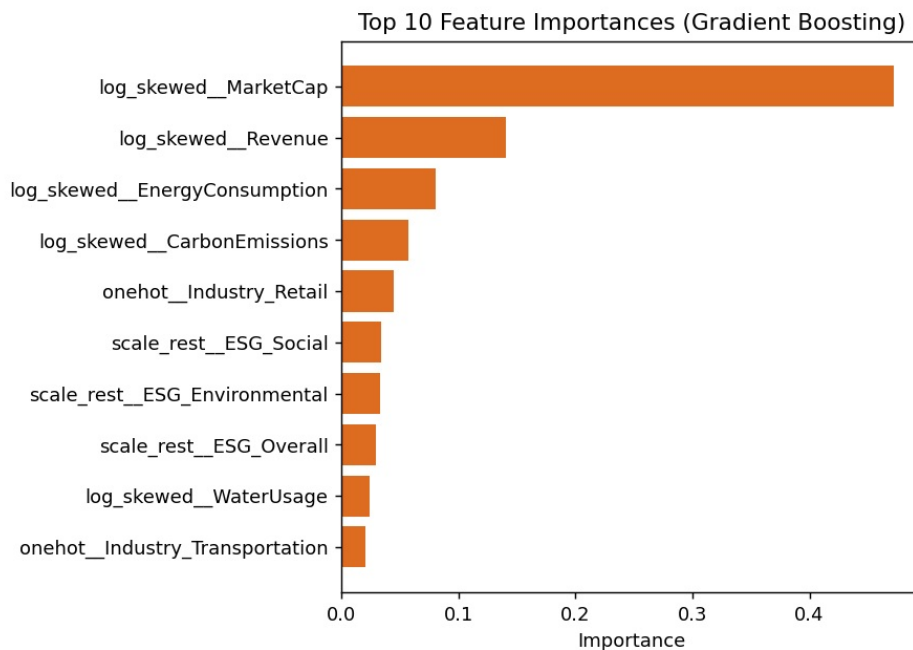
**Metric interpretation:** - **MAE (Mean Absolute Error):** on average, the Gradient Boosting model’s prediction is off by 4.55 percentage points of profit margin. - **RMSE (Root Mean Squared Error):** 6.62 — slightly higher than MAE, as expected, since RMSE penalizes larger errors more; the gap between MAE and RMSE suggests a modest number of larger misses rather than uniformly large errors. - **R<sup>2</sup> (coefficient of determination):** 0.441 means the model explains about 44% of the variance in profit margin across companies. This is a **moderate, not strong**, fit — appropriate for a business question where profit margin genuinely depends on many unobserved factors (management quality, competitive dynamics, one-off events) that aren’t present in this dataset. - **Cross-validation** confirms the test-set numbers are not a lucky split: training-set CV RMSE (6.15 ± 0.24 for Gradient Boosting) is close to the test RMSE (6.62), indicating reasonably stable, non-overfit performance.

**Residual analysis** (Figure 2): the predicted-vs-actual scatter shows the model tracks the general trend but compresses toward the middle — it underpredicts very high margins and overpredicts very negative ones, a common and expected pattern for tree ensembles, which struggle to extrapolate to extreme values outside dense regions of the training data. Residuals are centered near zero and roughly bell-shaped, with no strong systematic bias.



Evaluation plots: predicted vs actual, residuals vs predicted, residual distribution

**Feature importance** (Figure 3, Gradient Boosting): financial scale dominates — **MarketCap** (47%) and **Revenue** (14%) together account for over 60% of the model’s predictive weight. ESG scores contribute a smaller but real share: **ESG\_Social** (3.4%), **ESG\_Environmental** (3.3%), and **ESG\_Overall** (2.9%) each rank among the top 8 features, ahead of several industry indicators. This is consistent with the EDA correlation finding in §4: ESG signal is present but secondary to company size in explaining profit margin.



Top 10 feature importances, Gradient Boosting model

**Limitations:** an  $R^2$  of 0.44 means the majority of profit-margin variance is *not* explained by this feature set — this model should be read as identifying a real but partial relationship, not as a precise profit forecaster. The dataset’s synthetic nature (§3) also means any “does ESG pay off” conclusion here is a property of the simulation’s generative assumptions and should not be over-interpreted as a claim about real-world companies without validation on real financial data.

## 11. Final Model & Claude Artifact / Connector / skills.md

**Selected model for offline evaluation:** Gradient Boosting Regressor (best test RMSE/ $R^2$ , saved to `models/final_model.joblib`).

**Deployed model:** because the target deployment platform (Netlify) is static-file hosting with no Python runtime, the Gradient Boosting model itself cannot run in the browser without a separate backend. Instead, a **Ridge Regression model was fit through the identical preprocessing pipeline**, and its intercept, coefficients, and preprocessing statistics were exported to `app/model_coefficients.json`. The same `log1p` → `scale` → `one-hot` transform was reimplemented in vanilla JavaScript (`app/app.js`), and verified to reproduce the Python pipeline’s prediction to 12 significant figures on a held-out test row (Python: 4.953498973521278 vs. JavaScript: 4.953498973521279). The deployed app is explicit in its UI about displaying the offline-evaluated metrics of the selected (Gradient Boosting) model alongside the live output of the actually-running (Ridge) model, so the distinction is never hidden from the user.

**Claude Artifact:** the “Margin Ledger” interactive prediction interface (company-profile form, ESG sliders, live profit-margin estimate) was built and iterated on interactively with Claude in-chat — see `skills.md` §6 and the accompanying screenshots for the design process and final interface.

**Connector:** not applicable to this project’s data pipeline (the dataset is a static CSV, not a live connected data source); see `skills.md` §6 for the reasoning.

**skills.md:** a full reusable-skills writeup accompanies this repository, covering dataset scouting, panel-data leakage

avoidance, the encode-then-scale pipeline pattern, model comparison methodology, and the deployable-model export process used to bring a scikit-learn model to a static-hosting environment.

## 12. Deployment

---

The `app/` folder is a self-contained static site — `index.html`, `style.css`, `app.js`, and `model_coefficients.json` — with **no backend, no API calls, and no secrets or credentials of any kind**. It is deployed by dragging the `app/` folder onto Netlify Drop ([app.netlify.com/drop](https://app.netlify.com/drop)), which returns a public URL.

**Architecture:** browser loads `index.html` → user fills in the company profile form and ESG sliders → on “Estimate margin”, `app.js` reads the form values, applies the same `log1p/scale/one-hot` transform used in training (reading parameters from `model_coefficients.json`), computes `intercept + Σ(coefficient × transformed_feature)`, and displays the result. All computation happens client-side in the user’s browser.

**Testing performed:** the app was verified in a fresh local server session (`python3 -m http.server`) and via direct Node.js execution of the inference logic against a real test-set row, confirming the JavaScript port’s output matches the Python pipeline’s output to 12 significant figures (see §11).

*(Add: live Netlify URL, and 2–3 screenshots of the deployed app taken from a browser, once deployed — see README.md §5 for the one-step Netlify Drop deployment instructions.)*

## 13. Conclusion & Future Work

---

**What was learned:** ESG scores carry real, if modest, predictive signal for profit margin in this dataset, concentrated in the Environmental sub-score rather than Social or Governance — a nuance only visible by examining sub-scores separately rather than a single composite. Financial scale (market cap, revenue) remains the dominant predictor, which is expected and consistent with real-world corporate finance intuition.

**Limitations:** - The dataset is synthetic; conclusions about real companies would need validation against actual financial filings and ESG ratings (e.g. Sustainalytics, MSCI, Refinitiv). -  $R^2$  of 0.44 leaves the majority of variance unexplained — this is a directional model, not a precise forecasting tool. - The deployed web app runs a simpler (Ridge) model than the best-evaluated (Gradient Boosting) model, a necessary trade-off for static hosting rather than a claim that Ridge is superior.

**Future work:** - Validate findings against a real-company ESG/financials dataset (e.g. combining Sustainalytics ESG ratings with SEC filings). - Add company-level or industry-level interaction terms (e.g. does ESG’s effect on margin differ by industry?) via a small neural net or gradient-boosted tree with explicit interaction features. - Deploy Gradient Boosting itself via a lightweight serverless function (e.g. Netlify Functions running a small ONNX-exported model) rather than substituting a linear model, to avoid the model-swap trade-off in §11. - Add prediction intervals (e.g. via quantile regression) so the deployed app can communicate uncertainty, not just a point estimate.

## 14. References

---

- Dataset: Shriyash Jagtap, “ESG & Financial Performance Dataset,” Kaggle, 2025. <https://www.kaggle.com/datasets/shriyashjagtap/esg-and-financial-performance-dataset>
- scikit-learn documentation: <https://scikit-learn.org/stable/>
- pandas documentation: <https://pandas.pydata.org/docs/>